



SERVER-DRIVEN MODDING

Une nouvelle architecture de modding Minecraft

Le serveur comme plateforme · Le client vanilla comme terminal universel

Recherche technique exploratoire — protocole, JVM, runtime, sécurité

Baseline protocole : 26.3-snapshot-7 · Release : 26.2 · Août 2026

Niveau 0 = 85 % des usages sans installation · LinkAgent $\approx$ 300 Ko = $\sim$ 98 %

Sommaire

Sommaire	2
CONCLUSION DE RECHERCHE & PLAN DE CONSTRUCTION — Projet SDM	3
1. Conclusion de recherche — le résumé en une page	4
1.1 Ce qui a été démontré (3 documents, 1 laboratoire)	4
1.2 L'énoncé final de la rupture	4
1.3 Le principe de session sandbox (réponse à la question des keybinds/HUD)	4
2. Méthodologie de travail pour mener le projet à bien	6
2.1 La doctrine : « le seul échec possible est l'exécution »	6
2.2 Ingénierie — les disciplines non négociables	6
2.3 Process — itération en 3 boucles	6
2.4 Gestion des risques pendant la construction	6
3. Roadmap MVP puis complète	8
3.1 MVP — « la démo qui tue » (6 semaines, 1 dev)	8
3.2 Roadmap complète (après MVP)	8
4. Fonctionnalités et petits plus non cités jusqu'ici	10
4.1 Capacités plateforme (les « super-pouvoirs »)	10
4.2 Outils développeur (le DevKit)	10
4.3 Petits plus « détail qui vend »	10
5. La rupture appliquée à CocoWorld — concepts de conversion	12
5.1 Économie & Baltop (CocoEssentials)	12
5.2 Grades (chaîne LuckPerms)	12
5.3 Bosses (Cœur Gardien 90 %, Roi 0.2 %)	12
5.4 CocoCrops & CocoFishing	12
5.5 Guerre & civilisations (designs en cours)	12
5.6 Nether / Kronn (monde corrompu)	12
5.7 Lore : auras de Mack & Gorau	13
5.8 Collections & récompenses	13
5.9 File d'attente & hub (CocoQueue, Velocity)	13
5.10 StellarProtect	13
5.11 Priorisation des conversions (impact/effort)	13
6. Ce qui vient après la rupture (vision)	14

CONCLUSION DE RECHERCHE & PLAN DE CONSTRUCTION — Projet SDM

Document de clôture. La recherche est terminée (verdict : CONDITIONAL GO, validation sdm3). Ce document conclut puis transforme : méthodologie de travail, roadmaps, fonctionnalités, et application concrète à CocoWorld.

1. Conclusion de recherche — le résumé en une page

1.1 Ce qui a été démontré (3 documents, 1 laboratoire)

Rapport	Ce qu'il établit
sdm/ (44 p.)	Le plafond vanilla (N0 = 85 %), le mur des 5 familles, l'agent LinkAgent, SDMP, sécurité, roadmap
sdm2/ (22 p.)	La rupture UX : 7 verrous du zéro-clic, <code>JDK_JAVA_OPTIONS</code> , attach à chaud, version fantôme, architecture V0-V3
sdm3/ (23 p.)	Falsification : lab JDK 25 réel (env-var → premain → transformation ; attach → retransform live), matrice H1-H14, CONDITIONAL GO

1.2 L'énoncé final de la rupture

SDM déplace l'installation du modding du joueur vers la plateforme. Le serveur devient la source de tout (contenu, code, versions, signatures) ; le client devient un terminal universel (vanilla pur = 85 % ; + 1 clic Store une fois dans la vie = ~98 %). Le seul mode d'échec restant est l'exécution (code stupide, sécurité bâclée) — jamais la théorie.

1.3 Le principe de session sandbox (réponse à la question des keybinds/HUD)

Affirmation : tout ce qu'un serveur ajoute au client disparaît quand on quitte le serveur.

Architecture qui le garantit :

```

COUCHE SESSION (scopée à la connexion, vidée à la déconnexion)
├─ modules actifs (HUD, entités, rendu)
├─ keybinds virtuels (couche d'input dynamique, JAMAIS le registry vanilla)
├─ resource packs serveur (pop à la déconnexion)
├─ settings overrides (FOV, gamma... snapshotés → restaurés)
└─ état SDMP (cookies, manifeste, permissions actives)

COUCHE AGENT (permanente, générique, dormante)
├─ hooks génériques installés (input layer, Gui.render, event bus)
└─ sans session active : hooks inertes → comportement 100 % vanilla

```

Conséquences vérifiables :

1. Keybinds : l'écran options vanilla ne montre QUE le vanilla, toujours. Les binds SDM (configurés via l'écran SDM) n'existent que pendant la connexion au serveur qui les demande.
2. HUD : l'injection est un hook générique ; à la déconnexion il itère une liste vide → aucun pixel dessiné.
3. Crash mid-session : au boot suivant, aucun handshake → aucun module → vanilla propre. Le pire cas ne laisse jamais d'état sale.
4. Limite documentée : les classes chargées restent en RAM (limite JVM) mais comportementalement neutres — et le pattern « hook générique unique + tout le reste data-driven à runtime » minimise même ce résidu.

Règle d'architecture n°1 du projet : aucun module ne mute l'état vanilla directement. Tout passe par la couche session du runtime. C'est ce qui rend la promesse « sortir du serveur = comme si on n'y avait jamais été » vraie par construction plutôt que par nettoyage.

2. Méthodologie de travail pour mener le projet à bien

2.1 La doctrine : « le seul échec possible est l'exécution »

Tout le risque est concentré dans la qualité du code et la confiance. La méthodologie est donc construite autour de deux axes : ne jamais écrire de code stupide, et être irréprochable.

2.2 Ingénierie — les disciplines non négociables

1. Un repo, CI dès le jour 1. Build + tests automatiques à chaque commit ; le pipeline reproduit le lab (E1/E2) sur un JVM matrix (8/17/21/25) pour que les mécanismes ne régressent jamais silencieusement.
2. Le lab comme test de non-régression. Les expériences E1/E2 deviennent des tests JUnit exécutables : si `env-var` → `premain` → `transform` casse sur un JDK futur, le CI le crie avant tout joueur.
3. TDD sur les points critiques. Vérification de signatures, validation de manifests, quoting des chemins (le piège du lab), gestion de cache : écrits test-first. Ce sont les fonctions où un bug = perte de confiance existentielle.
4. Le pattern « hook générique unique ». Chaque classe vanilla n'est transformée qu'UNE fois, par un hook générique stable ; tout le spécifique vit en data/runtime. Cela réduit la surface de recettes par version de ~95 % et rend la maintenance multi-versions tractable.
5. Recettes versionnées + hash d'ancre + fallback N0 obligatoire. Une recette sans chemin de dégradation propre n'est jamais mergée (règle de review).
6. Revue par paire avec soi-même : le « audit du pire ennemi ». Chaque PR décrit ce qu'un attaquant pourrait faire avec ; le merge exige la réponse.
7. Tout open-source dès le départ (agent + service + protocole). La transparence est une fonctionnalité de sécurité et le meilleur argument marketing.
8. Jamais de deadline sur la sécurité. Phase 4 (signatures, permissions, CRL) slip si besoin — un incident de sécurité tue le projet, un slip ne tue rien.

2.3 Process — itération en 3 boucles

Boucle quotidienne (1 dev) : code → CI → test manuel rapide (serveur dev local)
 Boucle hebdo (J-passerelle) : démo jouable sur le serveur Canvas de dev, video logguée, retro perso
 Boucle par phase : critères de réussite de la roadmap = gate ;
 pas de phase suivante sans preuve (logs/vidéo)

2.4 Gestion des risques pendant la construction

Risque	Signal d'alerte précoce	Réponse préparée
JEP 451 (attach opt-in) devient défaut	notes de release JDK	bascule sur premain env-var (déjà rail principal) ; rien ne casse
Revue Store refuse	rejet de soumission	exe EV signé + version fantôme lancer-tier (canaux déjà spec)
Recette casse sur snapshot MC	hash d'ancre mismatch en CI	dégradation N0 automatique, recette re-ancrée (budget temps dédié)
Module malveillant tenté	télémetrie anormale	CRL + révocation clé + post-mortem public

Risque	Signal d'alerte précoce	Réponse préparée
Mojang réagit	mail/DMCA	position préparée : identique Fabric, zéro asset, opt-out ; ouvrir le dialogue

3. Roadmap MVP puis complète

3.1 MVP — « la démo qui tue » (6 semaines, 1 dev)

Objectif : prouver la chaîne complète sur Minecraft réel, en public filmable.

Sem.	Jalon	Livrable	Critère de réussite
S1	J1 — Consentement	Plugin Canvas : dialog « ★ Activer l'expérience » + <code>sdm:hello</code> login query + cookie de mémorisation	joueur vanilla voit le dialog, Oui/Non mémorisé
S2	J2 — Magie en session	Agent réel sur MC : recette <code>Gui.render</code> → HUD « SDM ✓ » apparaît en session live (env-var + attach)	vidéo : HUD apparaît sans restart, 0 crash
S3	J3 — Bidirectionnel	Bouton HUD cliquable → event C→S → commande serveur exécutée ; + tests de passage (mauvais hash, module absent)	chemin retour prouvé, dégradations propres
S4	J4 — Un vrai module	Module « Boss bar custom » : barre de vie stylée CocoWorld + phase announcements (remplace bossbar vanilla)	un feature visible, utile, désirable
S5	J5 — Vocal	Module voicechat : micro/opus/transport tunnelé	parler sans rien installer
S6	J6 — Session sandbox	Keybind virtuel + HUD + settings snapshot/restored à la déconnexion	quitter le serveur = retour vanilla vérifié

Le MVP est réussi si la vidéo de S2-S5 existe et qu'aucun crash n'est survenu. C'est le matériel de lancement (annonce, Discord, premiers serveurs pilotes).

3.2 Roadmap complète (après MVP)

Phase	Durée est.	Objectif	Sortie
P1 Runtime v1	4-6 sem	agent prod : premain+attach+cache+verify+session layer	bêta fermée agent
P2 SDMP complet	3 sem	hello/manifest/events/delta-sync/rollback	protocole figé v1
P3 Modules	4 sem	classloaders, lifecycle, hot-load/unload, isolation	10 load/unload sans fuite
P4 Sécurité	4 sem	Ed25519, permissions, consentement par clé, CRL, audit interne	THREAT-MODEL publié
P5 Server API	4 sem	ModuleHost, EventBridge, PackStudio	3 features démo
P6 Client caps	6 sem	HUD framework, keybinds, entités custom, rendu, vocal prod	module SDK client

Phase	Durée est.	Objectif	Sortie
P7 DevKit	4 sem	ServerModAPI + templates + docs + hot-reload dev	un dev externe écrit un module seul
P8 Store + distribution	3 sem	app MSIX, exe EV, fantôme publiée, site	installation publique 1 clic
P9 Compat	6 sem	adapter Fabric server-side, extraction assets	1 mod réel chargé côté serveur
P10 Production	continu	bêta ouverte, 3-10 serveurs pilotes, télémétrie, support	100 joueurs, 1 semaine, 0 incident

Total MVP→P8 : ~6 mois à 1 dev ; ~3 mois à 2 devs.

4. Fonctionnalités et petits plus non cités jusqu'ici

4.1 Capacités plateforme (les « super-pouvoirs »)

1. Cinématiques interactives — caméra scriptée + letterbox + bullet-time (N0) + overlays custom (N3) : intros de mise à jour, deaths cinématiques de boss.
2. Director musical adaptatif — couches audio mixées selon la situation (combat/exploration/événement), poussées par le serveur en packs.
3. Photo mode intégré — freeze + step + filtres GLSL streamés : screenshot partagé = marketing organique.
4. Killcam / replay théâtre — le serveur garde les N dernières secondes de position des entités ; re-diffusion caméra libre après un kill de boss.
5. Accessibilité pilotée par le serveur — palettes daltoniennes en packs, échelles d'UI par module, sous-titres custom des événements sonores.
6. A/B testing de contenu par joueur — deux variantes d'un module (prix, difficulté, UI) servies à deux cohortes ; métriques côté serveur.
7. Emotes système — animations partageables via display entities (N0) + bones custom (N3).
8. Nameplates custom — badges de grade, titres saisonniers, icônes de rôles rendus par module.
9. Écrans de mort / connexion custom — branding serveur jusqu'au moindre écran.
10. Widgets économie — portefeuille HUD, historique de transactions animé, previews d'items en 3D (ItemBody).
11. File d'attente vivante — position temps réel, ETA, mini-jeu pendant la queue (CocoQueue +).
12. Overlays streamer — le module expose des widgets OBS-friendly (séparés du HUD joueur) : le serveur devient stream-ready.
13. Cache LAN familial — deux joueurs derrière la même box partagent le cache des modules (dl 1x).
14. Mode spectateur staff enrichi — outils d'inspection (inventaires, historiques) en overlays sans changer le client.
15. Événements monde schedulés — saisons, world bosses, marchés itinérants : activation/désactivation horodatée, hot-load à l'heure pile.

4.2 Outils développeur (le DevKit)

16. Hot-reload modules en dev — modifier → re-push → testé en 5 s sans relancer le client.
17. Inspector in-game — F3-like SDM : état des modules, events traversants, erreurs, en overlay.
18. Éditeur de shaders live — tweak GLSL + re-push PostEffects sans reconnexion.
19. Replay de bugs — le client agent logge les events ; le dev rejoue la session du joueur qui a report.
20. Scaffold CLI — `sdm new module` → template compilé vers module+pack+recettes.

4.3 Petits plus « détail qui vend »

21. Toast de bienvenue custom au premier join équipé (« Experience complète activée ✓ »).
22. Migration douce : joueur vanilla voit en N0 ce que les équipés voient en N3 → teasing naturel.
23. Compteur « joueurs équipés » public sur le site (preuve sociale).

5. La rupture appliquée à CocoWorld — concepts de conversion

Principe : les plugins restent le cerveau (logique, vérité, persistance) ; les modules deviennent la peau (rendu, HUD, feedback). Communication par EventBridge (`sdm.event.*`). Voici le concept de conversion pour chaque pilier CocoWorld.

5.1 Économie & Baltop (CocoEssentials)

- Reste plugin : soldes, transactions, Baltop data, interests.
- Module : widget portefeuille animé (gain = +X flottant), écran Baltop 3D avec items en rotation, sons de transaction custom, graphiques de fortune dans un dialog enrichi N3.
- Dream modding : effets de pièces 3D sur les trades rares, prix du marché affichés en hologramme au hub.

5.2 Grades (chaîne LuckPerms)

- Reste plugin : permissions, chaînes de promotion.
- Module : nameplates par grade (icônes animées), auras de prestige (feu d'artifice de particules au promote), écran de progression de grade cinématique, cosmétiques liés au grade servis sélectivement (per-player modules).
- Dream : le grade VISUELLEMENT visible partout — chat custom, tab list stylée, emotes débloquent.

5.3 Bosses (Cœur Gardien 90 %, Roi 0.2 %)

- Reste plugin : phases, dégâts, loot tables (Cœur Gardien, Roi).
- Module : barre de boss custom (art CocoWorld, pas la barre vanilla), télégraphes d'attaques au sol, annonces de phase fullscreen, killcam à la mort, révélation de loot cinématique (l'item tombe en 3D, lumière, son).
- Dream : le boss Roi comme événement serveur — annonce à tous, HUD de participant, drop 0.2 % célébré serveur-wide en overlay.

5.4 CocoCrops & CocoFishing

- Reste plugin : croissance, récoltes, séries, stats de pêche.
- Module : croissance animée (interpolation display N0 → vraies plantes 3D N3), minigame de pêche (jauge de tension custom, écran dédié), poissons rares en modèles 3D avec introduction cinématique à la capture.
- Dream : météo qui influence visuellement les cultures (pluie = croissance visible accélérée en particules).

5.5 Guerre & civilisations (designs en cours)

- Reste plugin : territoires, scores, villager AI (serveur).
- Module : carte de territoire HUD (minimap custom), murs de sélection de région en particules (StellarProtect +), bannières animées, cinématiques de siège, emploi du temps des villageois visualisé (N3).
- Dream : le « war room » — écran de commandement temps réel pour les chefs.

5.6 Nether / Kronn (monde corrompu)

- Reste plugin/datapack : worldgen, structure, progression.
- Module : shader de corruption progressif (PostEffects piloté par la progression du joueur — plus il s'enfonce, plus l'écran se corrompt), couches audio de détresse, hallucinations (fausses entités éphémères), heartbeat HUD près de Kronn.
- Dream : la corruption est une expérience sensorielle, pas un biome.

5.7 Lore : auras de Mack & Gorau

- Module : rendu d'aura continu (particules GPU, pas des particules serveur = zéro lag), aura blanche Gorau ×1000 comme événement visuel, effets d'armes légendaires.
- Dream : les moments lore (mort de Gorau, ère 315) rejouables en « chroniques » cinématiques dans le Berceau musée.

5.8 Collections & récompenses

- Module : musée de collection personnel (pièce 3D, items en vitrine rotative), toasts de succès custom (pas les advancements vanilla), progression saisonnière (passe) avec récompenses visuelles.

5.9 File d'attente & hub (CocoQueue, Velocity)

- Module : widget queue avec ETA + mini-jeu, écran de transfert de serveur custom (au lieu du vide), cohérence visuelle hub → mondes.

5.10 StellarProtect

- Module : visualisation de régions (murs de particules au claim), indicateurs de confiance, mode inspection staff.

5.11 Priorisation des conversions (impact/effort)

Vague	Modules	Pourquoi
1 (MVP+)	Boss custom bar, wallet widget, toasts	petits, très visibles, prouvent le style
2	Auras lore, shader Nether, nameplates grades	l'identité CocoWorld visuelle
3	Pêche minigame, musée collections, war map	gameplay profond
4	Killcam, war room, chroniques	le « wow » marketing

6. Ce qui vient après la rupture (vision)

1. Court terme : CocoWorld = serveur vitrine (« le premier serveur où le modding s'installe tout seul ») → contenu marketing natif.

2. Moyen terme : 3-10 serveurs pilotes FR → la plateforme devient un produit (inscription, registry de modules signés, dashboard owner).

3. Long terme : le standard de distribution — les créateurs publient des modules SDM comme ils publient des plugins aujourd'hui ; le registry devient le « Modrinth du server-driven » ; la position d'infrastructure se défend par la confiance et l'outillage, pas par un walled garden.

La thèse de sortie de recherche, une dernière fois : la révolution n'est pas technologique — la technologie est prouvée. La révolution est la suppression de la friction entre l'envie de jouer et le jeu. Cinq ans de recherche y ont répondu : maintenant, on construit.

RECHERCHE : TERMINÉE (CONDITIONAL GO)

PROCHAINE PIERRE : J1 — 1 semaine de code

CIBLE : MVP 6 semaines → première vidéo publique
